```
    "build:js":
      "esbuild app/javascript/*.* --bundle
       --sourcemap --outdir=app/assets/builds",
    "build:css":
       "tailwindcss -i ./app/assets/stylesheets/application.tailwind.css
       -o ./app/assets/builds/tailwind.css",
    "failure:js":
      "rm ./app/assets/builds/application.js &&
      rm ./app/assets/builds/application.js.map",
    "dev":
      "tsc-watch --noClear -p tsconfig.json --onSuccess \"yarn build:js\"
      --onFailure \"yarn failure:js\""
  }
}
```

The package.json file is used to manage dependencies, configure some of those dependent tools, define commands for working with your code, and store information about your code should you choose to publish your code as a node module in its own right. This is different from Ruby gems behavior, where the dependency management is in Gemfile.lock, but if you are packaging your code, the metadata goes in a .gemspec file and dependency configuration is done separately. (I'm not going to talk about the metadata that is only used when publishing a module.)

A lot of different information can be placed in a package.json file, but let's start with the information in the file we already have. This file was generated by Rails at startup, and we've added a few more modules to it as we've moved forward.

Our package.json file has five keys:

name
: This is the name of the module, assuming we are going to package it. Looks like the default value is app. I haven't changed it.

private
: This should be set to true in our case. You'd set it to false if you planned to publish the module.

dependencies
: The dependencies key has a list of packages that this code depends on to